POLITECNICO
MILANO 1863

ATD

Hu Peng – ID: 11032747
Zhang Zihao – ID: 11030184

Academic Year 2025–2026

Contents

1 Front Page

Project Name	Best Bike Paths (BBP)
Deliverable	ATD
Course	Software Engineering 2
Academic Year	2025–2026
Authors	Hu Peng (ID: 11032747)
	Zhang Zihao (ID: 11030184)
Version	1.0
Date	7 February 2026
Repository	`https://github.com/euorrl/HuZhang`

2 Project Identification

This Acceptance Test Deliverable concerns the project *Best Bike Paths (BBP)*, developed by Matteo Caldognetto and Claudia Salone during the Academic Year 2025–2026.

BBP is a community-driven web platform that enables cyclists to record trips, report path conditions, and discover routes based on aggregated community data. The system supports both manual data entry and simulated automatic route generation, and allows users to publish validated trips as shared community paths.

The Design Document (DD) describes a Cloud-Native, Serverless architecture deployed on an Edge infrastructure. The system follows a layered structure composed of a React-based Single Page Application (presentation layer), a Hono/tRPC backend (application layer), and a PostgreSQL database (data layer). External services are integrated for routing and environmental data enrichment.

The acceptance testing activity has been conducted with reference to the following artifacts:

- Requirement Analysis and Specification Document (RASD), Version 1.0
- Design Document (DD), Version 1.0
- Implementation and Testing Document (ITD), Version 1.0
- Source code repository available at: `https://github.com/matteocaldognetto/CaldognettoSalone`

The objective of this deliverable is to assess whether the implemented system satisfies the requirements defined in the RASD and whether its observed behavior is consistent with the architectural and implementation choices described in the DD and ITD.

3 Installation Setup

I did not find that the project has been deployed online, so I conducted local deployment testing based on the ITD document.

3.1 Prerequisites

In ITD:

Tool	Version	Description
Bun	1.2.0	TypeScript runtime and package manager
Docker	Latest stable	PostgreSQL container for local development
Git	Any recent	Version control
Node.js	18+ (optional)	Not strictly required; Bun replaces Node.js

Table 1: Required software and tools in ITD

Test environment:

Tool	Version
Bun	1.3.8
Docker	29.2.0
Git	2.45.1
Node.js	24.13.0

Table 2: Software and tools in test environment:

3.2 Preparing

The preparation phase focuses on setting up the local development environment and installing all required project dependencies. All installation steps are performed using Windows PowerShell.

Repository cloning. The source code is retrieved from the official GitHub repository using Git:

```
git clone https://github.com/matteocaldognetto/CaldognettoSalone.git
```

Project directory setup. After cloning the repository, the working directory is set to the project root:

```
cd CaldognettoSalone/src
```

Dependency installation. All dependencies required by the monorepo are installed using Bun, which acts as both the runtime environment and the package manager:

```
bun install
```

This command installs the dependencies for all workspaces, including the frontend, backend, and shared packages. During the first execution, the installation process may take several minutes due to the download and setup of external dependencies.

At the end of this phase, the local environment is fully configured and ready for application execution.

3.3 Running

Once the preparation phase is completed, the application can be executed locally by starting its services individually. Given the monorepo structure of the project, each component is launched from its corresponding workspace.

Database service. The PostgreSQL database is started using Docker Compose. This operation is required only once, as the container runs persistently in the background:

```
docker compose up -d postgres
```

Backend service. The backend application is located in the `apps/api` workspace and is started using the following commands:

```
cd apps/api
bun dev
```

After startup, the backend listens for incoming requests on the port defined in the environment configuration (default: `http://localhost:8787`).

Frontend service. The frontend application is located in the `apps/app` workspace and is executed in a separate terminal window:

```
cd apps/app
bun dev
```

Once the startup process is completed, the frontend can be accessed via a web browser at:

```
http://localhost:5173
```

Execution order. To ensure correct communication between system components, the database and backend services must be started before launching the frontend.

At the end of this phase, the complete system is running locally and can be accessed through the web interface for functional testing and demonstration purposes.

4 Acceptance Test Cases and Results

This section presents the acceptance test cases designed to validate the correct implementation of the system functionalities described in the RASD document. All tests have been executed on the local deployment environment described in Section 3, using the integrated frontend, backend services, and the PostgreSQL database.

In ITD, the author wrote the functionalities and their implementation status. Next, we will try to verify these features.

Refer to the RASD document for detailed descriptions under section 2.A.1

Functionality	**Status**
Registration and authentication	Implemented
Automated route recording	Simulated
Manual route recording	Implemented
Path visualization between two points	Implemented
Display of the registered paths in the personal inventory	Implemented
Reporting another path	Implemented

Figure 1: Implemented functionalities and their implementation status in ITD

4.1 Registration and authentication

In non-registration mode, users can see the *Find Route* functionality and the option to register an account. Only route exploration features are available, while advanced functionalities are restricted.

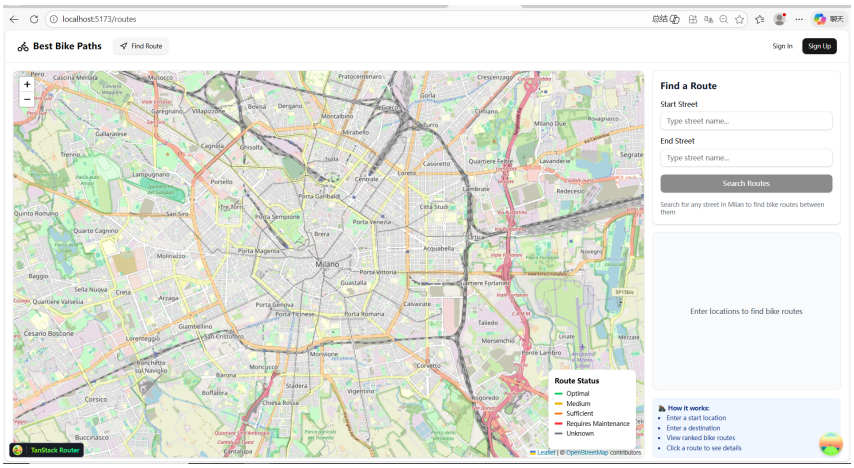

Figure 2: User interface in non-registration mode

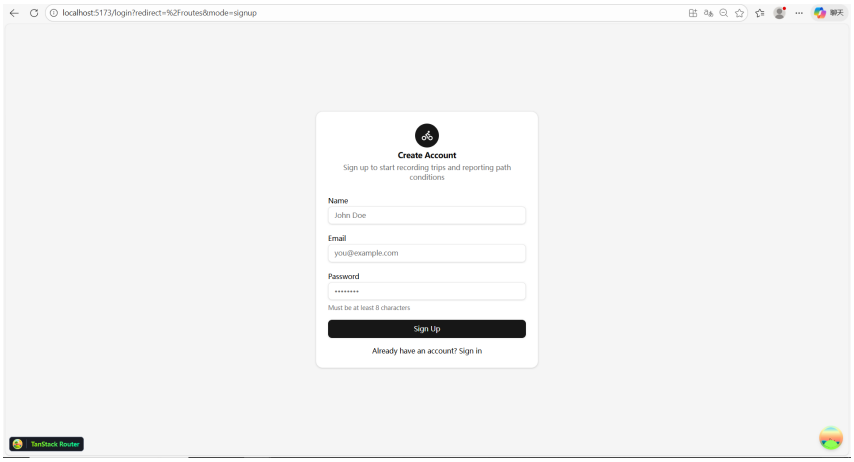

Figure 3: Registration interface

In registered mode, users can log in and access all the remaining functionalities of the system, including route recording, path management, and reporting features.

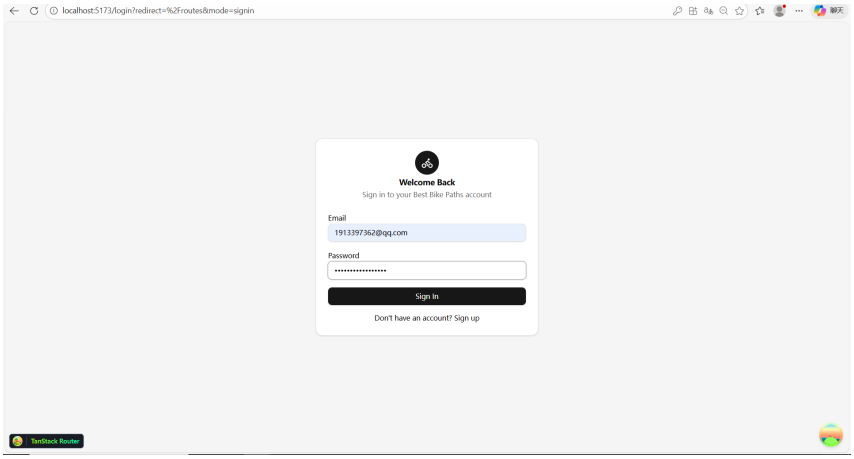

Figure 4: Login interface

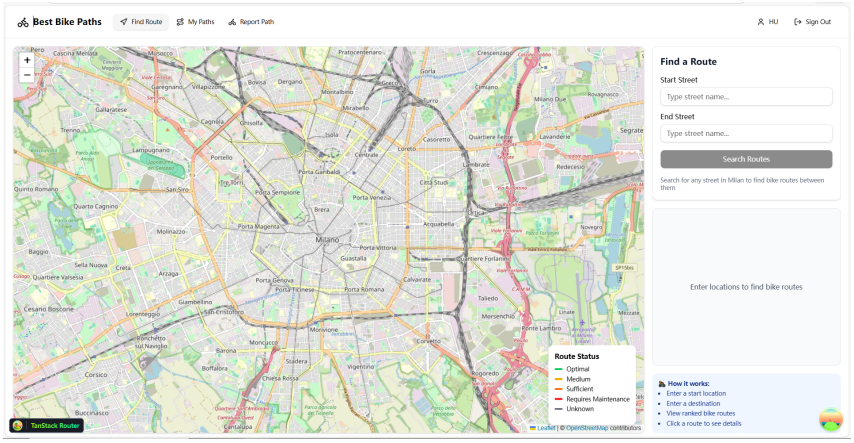

Figure 5: User interface in registered mode

4.2 Automated route recording

From the *My Paths* page, users can activate the *Automatic Mode* to simulate automatic route recording. After clicking the *Automatic Mode* button, the system automatically generates a route without requiring manual input of individual streets. Once the automatic mode is activated, the interface displays a modal window summarizing the generated route.

Figures 6 and 7 illustrate the user interface during the automatic route generation process and the resulting route summary.

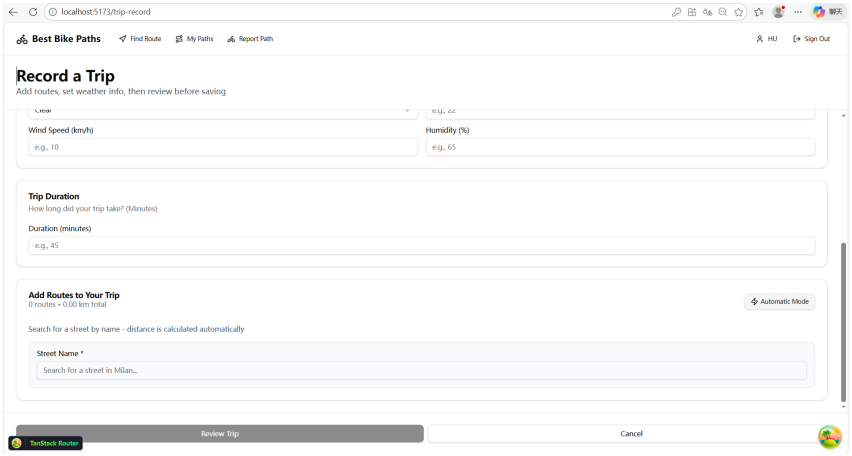

Figure 6: Automatic Mode from the My Paths page

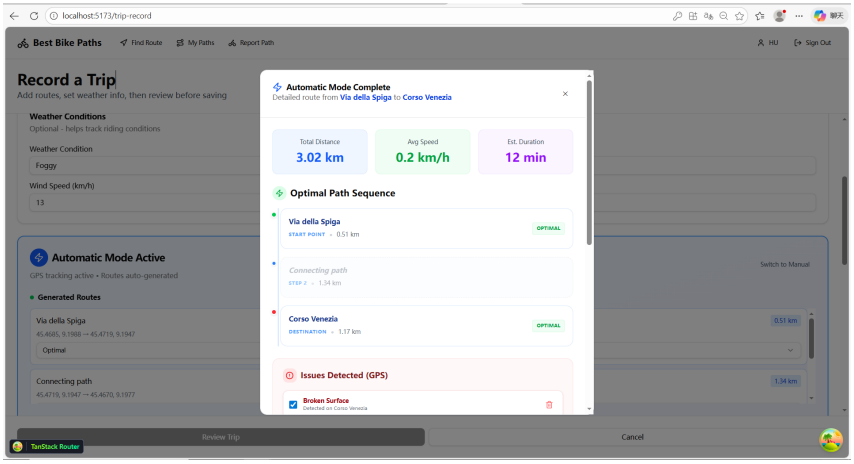

Figure 7: Path of the Automatic Mode

4.3 Manual route recording

This test case evaluates the behavior of the system when the user records a trip using the manual insertion mode.

In this mode, the system allows the user to manually provide descriptive information for the path, including the path name, textual notes, and weather conditions. These inputs are correctly accepted and stored as part of the trip description.

The riding duration is not automatically computed by the system and must be explicitly entered by the user. No automatic inference of time is performed during manual route recording.

When specifying the start and end points of the route, the system restricts the selection to a predefined set of streets. The origin and destination cannot be set based on the user's current GPS position, and free

selection of arbitrary coordinates is not supported in this mode.

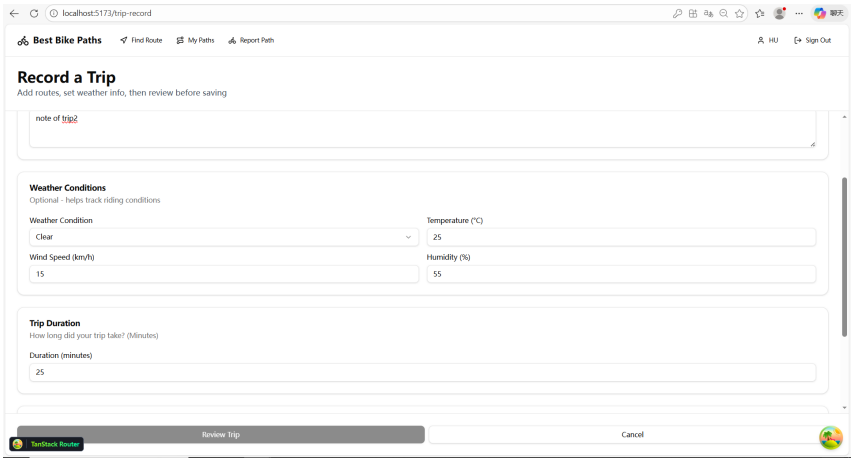

Figure 8: Manual Mode from the My Paths page

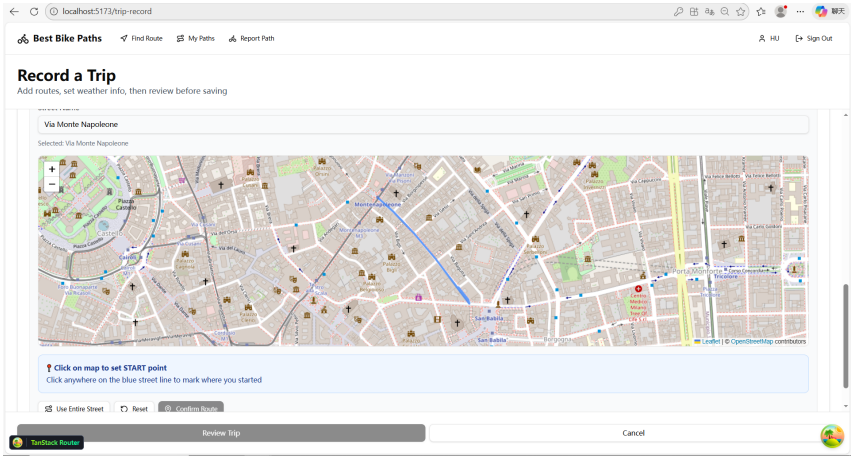

Figure 9: Set starting and ending points

4.4 Path visualization between two points

Visualization of the path between two points has been achieved, but the starting and ending points cannot be distinguished.

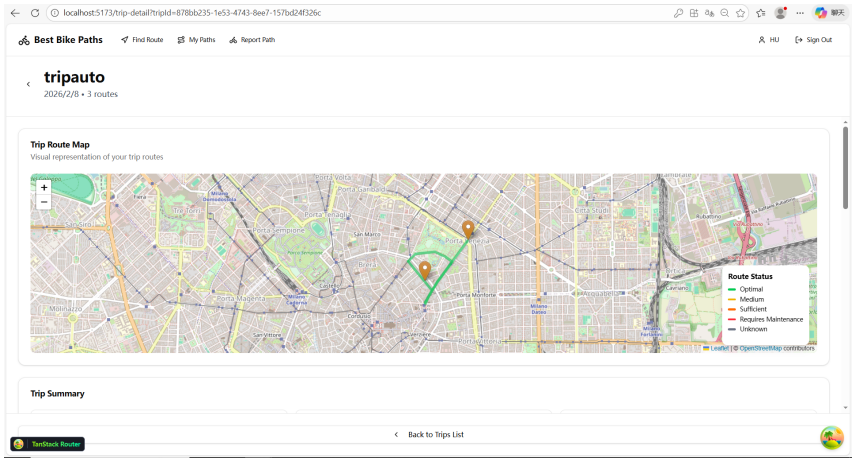

Figure 10: Path visualization between two points

4.5 Display of the registered paths in the personal inventory

The recorded paths can be viewed in My Paths

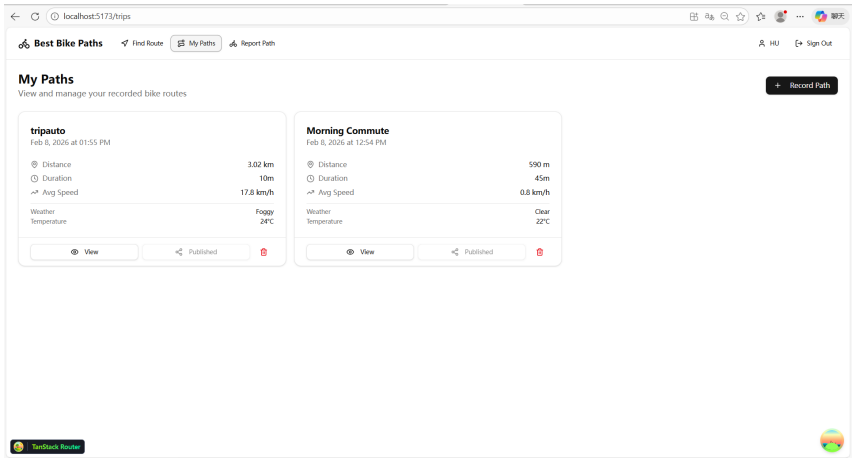

Figure 11: Display of the registered paths in the personal inventory

4.6 Reporting another path

I can add some information and publish it on the Report Path page, and then find the trip on the Find Route page.

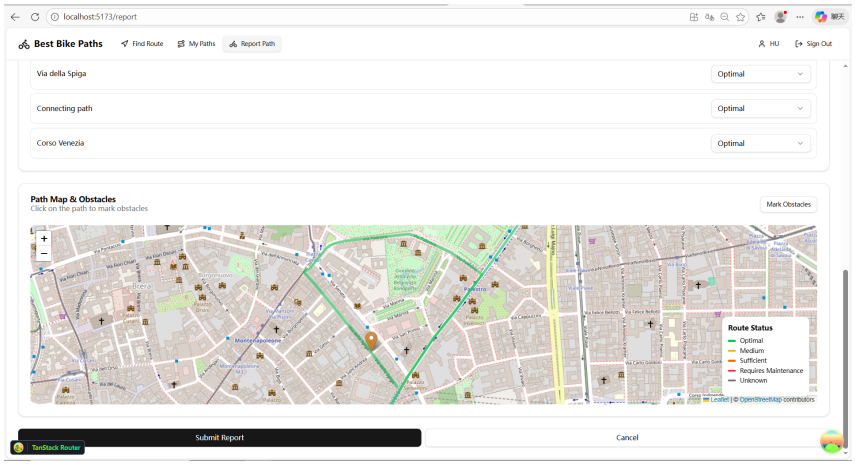

Figure 12: Reporting path

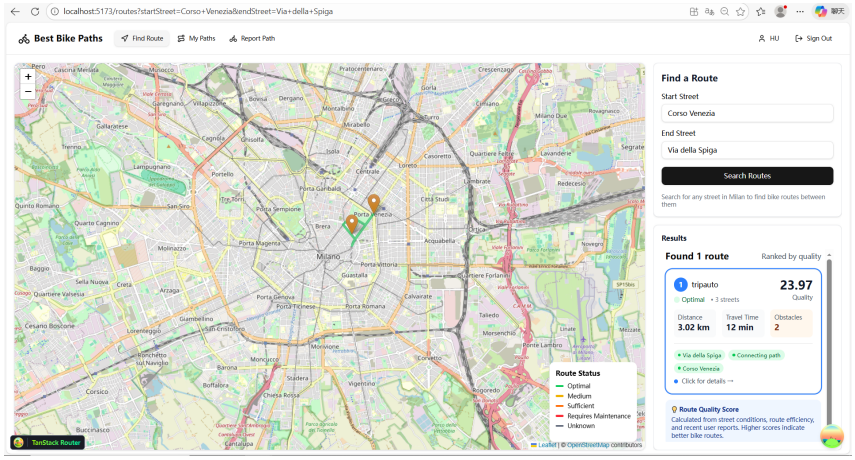

Figure 13: Find Route

5 Documentation and Code Quality Remarks

Based on the execution of the acceptance tests and the observed system behavior, several potential improvements can be identified.

First, the system does not currently support positioning based on the user's real-time location. Routes cannot be dynamically generated or updated according to GPS data during the trip. Introducing real-time location tracking would allow the system to automatically record paths and provide a more realistic riding experience.

Second, the duration of each trip is manually entered by the user. This approach may lead to inaccurate or inconsistent data. A more robust solution would be to let the system automatically record the riding time based on the actual start and end of the trip.

Third, the selection of start and end points is limited to a predefined set of streets. Users cannot freely define arbitrary start or destination locations, nor can they use their current position as a reference. In addition, the names of the start and end points are not explicitly stored. Recording both the coordinates and the corresponding street names would improve traceability and usability.

Moreover, when visualizing routes, the system does not distinguish the start and end points through different map icons. Using distinct markers for the origin and destination would improve clarity and readability of the route visualization.

Another limitation concerns route discovery. Since the system does not store explicit start and end point names, and the *Find Route* functionality requires street names as input, it becomes difficult for users to retrieve routes that they have previously recorded and published.

Finally, the system does not provide an overview of which paths are already available for querying. Users have no indication of existing routes unless they know the exact input parameters. Providing a list or map-based overview of available paths would significantly enhance the user experience.

Overall, addressing these aspects would improve automation, usability, and consistency of the system.

6 Effort Spent

The following table displays the number of hours each group member spent on the various sections of the document. Please note that this division is only approximate, and each section still required the collaboration of all members.

Table 3: Distribution of effort among group members

Member	Chapter 1	Chapter 2	Chapter 3	Chapter 4	Chapter 5
Hu Peng	1	2	4	5	4
Zhang Zihao	1	3	3	4	3

References

